

Stochastic Residual Dynamics for Latent World Models: Flow-Matched Residual Kernels on a Frozen LeWorldModel

Feiyang Wu Jiayi Wang
School of Electrical and Computer Engineering
Georgia Institute of Technology
{fwu91, jwang3617}@gatech.edu

Abstract

Joint-embedding predictive architectures such as LeWorldModel (LeWM) learn compact latent dynamics that make planning from pixels cheap, but their predictor is a deterministic point map. Under episode-constant hidden physics — friction, mass, contact compliance — the true next-state conditional is multimodal, and a point predictor recovers a conditional mean that can lie in the low-density valley between physical outcomes. We freeze a trained LeWM and learn a conditional flow-matched residual kernel on top of it, turning the deterministic transition into a calibrated stochastic kernel cheap enough for particle MPC. We grade the kernel against ground truth obtained from an exact simulator. The residual kernel improves exact-fork energy score by 34.8% over deterministic LeWM (3 seeds, $H=10$) and by 20.2% over its own predictive mean, at zero cost to the base model’s point predictions. At the control level the picture is sharper than a single number: ordinary receding-horizon MPC shows no significant gain, while a task that forces commitment under hidden friction shows +3.8 pp simulator success from a label-free persistent-mode residual, against a +22 pp context-level ceiling established by an exact-fork oracle.

1 Introduction

World models learn predictive representations of an environment that support planning, policy learning, and imagined rollouts [1]. Latent-space formulations are attractive for robotics because planning directly in observation space is expensive and pixel prediction wastes capacity on visually detailed but control-irrelevant structure. Joint-Embedding Predictive Architectures (JEPAs) take this to its conclusion: predictions are compared in representation space, and nothing is reconstructed [6–9]. LeWorldModel (LeWM) is the current end-to-end instantiation of this idea for control — a ViT encoder and an autoregressive predictor trained from raw pixels with only two loss terms, a next-embedding prediction loss and a Gaussian latent regularizer, giving a $\sim 15\text{M}$ -parameter world model that plans far faster than foundation-model-based alternatives [10].

The predictor at the heart of LeWM is deterministic. For a given latent z_t and action a_t it returns a single next embedding. This is adequate in smooth regimes, but robot dynamics are not uniformly smooth: contact, friction, compliance, and partial observability make the one-step conditional $p(z_{t+1} \mid z_t, a_t)$ genuinely multimodal. A squared-error-trained point predictor can at best recover the conditional mean of a multimodal target — and the mean of a bimodal distribution sits *between* the

modes, in a region the system never visits. A planner that rolls out this mean does not merely lose accuracy; it optimizes against a physically impossible future, and can commit to an action that fails under every outcome nature might actually draw.

This is the gap we address: JEPA-style latent world models give efficient deterministic next-embedding prediction but do not model the conditional stochastic residual needed for uncertainty-aware rollout. Two adjacent lines suggest the remedy. Residual dynamics learning decomposes a transition into a nominal component and a learned correction [17–20], and recent work has moved that correction into latent world-model space rather than image space [21–23]. Flow matching trains continuous normalizing flows by regressing a velocity field along a probability path [11–14], and models multimodal continuous conditionals with few function evaluations at sampling time — exactly the cost profile a planner can afford inside an inner loop.

We combine the two. We leave the LeWM encoder and predictor entirely untouched, treat their output as a *nominal delta*, and learn a conditional flow-matched kernel over the whitened residual between the nominal delta and the realized transition. Because the nominal is frozen, the base model’s point-prediction quality is preserved by construction, and every gain we report is attributable to the added distribution. A nesting of baselines — zero residual, correctly-scaled Gaussian noise, conditional Gaussian, memoryless flow, memory flow — lets us attribute the benefit rung by rung rather than crediting the most sophisticated component with the whole gain.

Contributions

1. *A residual stochastic kernel for a frozen JEPA world model.* A conditional flow-matched velocity field over whitened residuals, optionally conditioned on a small GRU memory or split into label-free persistent modes, converting a deterministic latent predictor into a calibrated transition kernel at $\text{NFE} \in \{4, 8\}$.
2. *An exact-fork evaluation protocol.* Ground-truth conditional distributions obtained by forking the simulator state and resampling the hidden physical mode, so stochastic world models are scored with proper scoring rules against the true kernel — not only marginal likelihood or downstream return.
3. *A benefit ladder and an honest scope.* We show that most of the one-step distributional gain comes from acknowledging noise at the correct scale, that learned conditioning adds the rest, and that memory acts as adaptive recent-context conditioning rather than a Bayes filter over hidden modes.
4. *A characterization of when stochasticity reaches control.* Prediction gains do not automatically become success gains: receding-horizon re-planning hides the benefit, while commitment-under-uncertainty tasks expose a structural +22 pp ceiling that a distribution-aware planner can capture and a mean planner cannot.

1.1 Related work

Latent world models and JEPAs Recurrent latent dynamics models learn compressed state features for control from pixels [1]; PlaNet plans online in a learned latent space [2] and Dreamer

backpropagates value gradients through imagined latent trajectories [3], while MuZero shows that a model need not reconstruct observations if it predicts what planning consumes [4]. Reconstruction-free predictive architectures push this further by comparing predictions in representation space [6], instantiated for images [7], video [8], and embodied planning [9]; DINO-WM instead builds world models on frozen pretrained visual features [5]. LeWM [10] is the end-to-end variant we build on. All of these predictors are trained with deterministic embedding losses, so under multimodal futures they can only return a conditional mean.

Flow matching Flow matching trains a velocity field to transport a base distribution to a target along a prescribed probability path [11], with closely related straight-path [12], conditional/minibatch-OT [13] and stochastic-interpolant [14] formulations. In robot learning, generative models of behaviour are now standard — diffusion policies [15] and flow-based policies [16] — but the generative machinery sits on the *action* distribution. We place it on the *transition residual* instead, which is what a planner needs in order to reason about outcomes rather than about its own action noise.

Residual and uncertainty-aware dynamics Uncertainty-aware dynamics learning for control dates to Gaussian-process models [17], with deep neural dynamics used for MPC in manipulation [18] and physics-structured models improving extrapolation [19, 20]. Closest to us, ReDRAW adapts a pretrained latent world model through residual corrections of latent-state dynamics [21], FlowMo-WM separates action-conditioned base dynamics from context-dependent drift [22], and RLA-WM predicts residual latent actions with flow matching [23]. Relative to these, our residual is (i) a full conditional *distribution* rather than a correction term, (ii) trained on a strictly frozen nominal, so the base model’s point accuracy is preserved by construction, and (iii) scored against ground-truth conditionals obtained from exact simulator forks rather than against realized trajectories alone.

2 Method

2.1 Preliminaries: the deterministic LeWM base model

Let o_t be an observation and a_t an action. The encoder E_θ maps an observation to a latent embedding $z_t = E_\theta(o_t) \in \mathbb{R}^d$, taken as the CLS token of a ViT-tiny at 224×224 with patch size 14 followed by a projector; in our runs $d = 192$. An action encoder A_θ embeds the frame-skipped action, and the autoregressive predictor P_θ — a six-layer causal transformer whose blocks are conditioned on the action embedding through AdaLN-zero modulation — produces a point prediction of the next embedding,

$$\hat{z}_{t+1} = P_\theta(z_t, \tilde{a}_t), \quad \tilde{a}_t = A_\theta(a_t).$$

Training minimizes LeWM’s two-term objective — a next-embedding prediction loss against a stop-gradient target plus a sketched isotropic-Gaussian regularizer that prevents representational collapse,

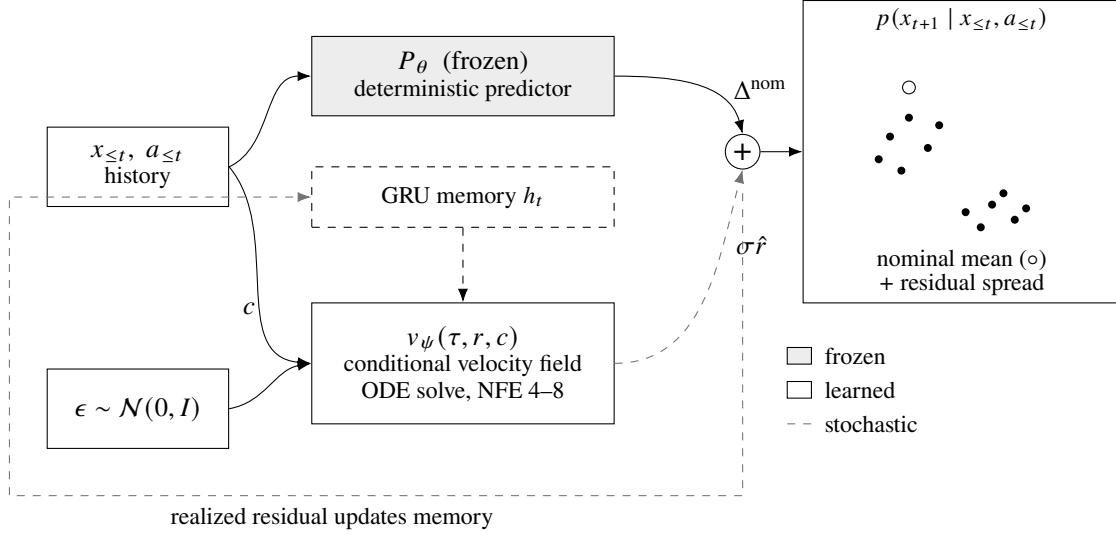


Figure 1: A frozen nominal, a learned residual, a small memory. The deterministic predictor P_{θ} (shaded) is frozen and its point prediction is untouched; it supplies the nominal delta Δ^{nom} . A conditional velocity field v_{ψ} transports a Gaussian sample ϵ to the whitened residual — what the environment’s hidden physics actually did — which is rescaled by the fixed diagonal statistics σ and added to the nominal prediction, giving a full conditional $p(x_{t+1} | x_{\leq t}, a_{\leq t})$ rather than a single point. A one-layer GRU carries recent residual context across the episode and is updated online from realized residuals. Because the flow’s output head is zero-initialized, the model starts as the deterministic world model exactly and can only add structure; setting $\sigma = 0$ recovers it at any time. Particle MPC (Sec. 2.6) draws N futures from this kernel with common random numbers.

$$\mathcal{L}_{\text{LeWM}} = \|\hat{z}_{t+1} - \text{sg}(z_{t+1})\|_2^2 + \lambda \mathcal{L}_{\text{reg}}(z),$$

with $\lambda = 0.09$ and the regularizer computed from an Epps–Pulley statistic over 1024 random one-dimensional projections at 17 quadrature knots. Control is latent MPC: roll P_{θ} over a horizon with history size 3 and minimize distance to the goal embedding.

2.2 Residual formulation and target whitening

We freeze the deterministic model — either the released LeWM checkpoint or a *task-adapted nominal*, i.e. vanilla LeWM retrained on the task’s own data — and learn a residual on top of it; P_{θ} below denotes that frozen predictor, whose parameters are never updated again. Let x_t be the dynamics variable: the latent z_t for the latent variant, or the 28-dimensional Fetch observation for the *deployable state variant* used in all FetchPush and FetchSlide control experiments (no privileged simulator state and no friction labels enter the model). Write the frozen nominal delta and the realized delta as

$$\Delta^{\text{nom}}(x_t, a_t) = P_{\theta}(x_t, a_t) - x_t, \quad \Delta_t = x_{t+1} - x_t.$$

Using train-only statistics — residual mean μ_r and diagonal scale σ , both hashed and immutable — the whitened residual target is

$$r_t = \sigma^{-1}(\Delta_t - \Delta^{\text{nom}}(x_t, a_t) - \mu_r).$$

Task adaptation matters here, and is the reason we run a dual-nominal design. On the quadruple-stack pilot the residual mean norm drops from 1.11 against the official nominal to 0.05 against the task-adapted one (mean diagonal scale $0.494 \rightarrow 0.028$), so the target r_t is close to pure aleatoric noise rather than systematic nominal bias. Any gain measured against the task-adapted nominal is therefore a gain in *stochasticity modeling*, not in bias correction.

2.3 Conditional flow-matched kernel

The kernel is a velocity field $v_\psi(\tau, r, c)$ — a four-layer LayerNorm/GELU MLP of width 512 with a 128-dimensional sinusoidal embedding of the flow time τ and a zero-initialized output layer — conditioned on $c = c(x_t, a_t, \Delta^{\text{nom}})$. With a Gaussian base sample $\epsilon \sim \mathcal{N}(0, I)$ and the linear interpolant $r_\tau = (1 - \tau)\epsilon + \tau r$, the flow-matching loss is

$$\mathcal{L}_{\text{FM}} = \mathbb{E}_{\tau \sim U[0,1], \epsilon, (x_t, a_t)} \left\| v_\psi(\tau, r_\tau, c) - (r - \epsilon) \right\|_2^2.$$

Sampling integrates the probability-flow ODE $\dot{r} = v_\psi(\tau, r, c)$ from $r_0 = \epsilon$ over $\tau \in [0, 1]$ at NFE $\in \{4, 8\}$, giving $\hat{r} = r_1$. The stochastic transition is

$$x_{t+1} = x_t + \Delta^{\text{nom}}(x_t, a_t) + \sigma \hat{r}, \quad \hat{r} \sim K_\psi(\cdot \mid x_t, a_t).$$

This construction is deliberately nested. Setting $\hat{r}$ to the kernel mean recovers a deterministic residual baseline; setting $\sigma = 0$ recovers the frozen LeWM exactly. The same interface is implemented by three simpler heads used as controls — a global diagonal Gaussian in whitened space (no learning, no conditioning), a conditional diagonal Gaussian, and a conditional Gaussian mixture — all sharing the same frozen residual targets and normalization statistics, so the ladder is a clean ablation rather than a comparison of differently preprocessed models.

2.4 Recurrent memory variant

Hidden physical modes are episode-constant, so observed transitions are informative about the future. A one-layer, 128-dimensional GRU carries a hidden state h_t updated online from the condition, the realized whitened residual and (optionally) a deployable low-dimensional observation delta; the kernel condition becomes $c_t = (x_t, a_t, \Delta^{\text{nom}}, h_t)$. The memory module is functional — the caller owns and passes the state — so candidate and particle axes cannot be mixed during planning. Training teacher-forces at most `history_size` - 1 = 2 updates; the diagnostics in Sec. 3.4 show the state nevertheless remains stable and useful out to 200 replayed updates.

2.5 Label-free persistent mode heads

For commitment tasks we replace the single kernel with two residual heads $g_{\psi}^{(1)}, g_{\psi}^{(2)}$ producing whitened residuals $r^{(k)}(c)$. Paired episodes that share an initial scene — known from data collection, but *without* friction labels — are assigned to opposite heads by a within-pair winner-take-all on object-motion dimensions, and each episode is trained under one persistent head assignment for its entire length:

$$x_{t+1}^{(k)} = x_t + \Delta^{\text{nom}}(x_t, a_t) + \sigma r^{(k)}(c), \quad k \in \{1, 2\}.$$

The matched deterministic baseline for this model is the head average $\bar{r} = \frac{1}{2}(r^{(1)} + r^{(2)})$, so the comparison isolates distribution-awareness from capacity. Persistent per-episode outcome identity — not one-step stochasticity — turns out to be the necessary ingredient (Sec. 3.3).

2.6 Particle-based model-predictive control

A candidate action sequence $a_{1:H}$ is evaluated by sampling $N = 8$ residual particles with common random numbers and scoring goal cost. Two selectors use the *same* kernel and differ only in whether they reason about the distribution:

$$a_{\text{det}}^{\star} = \arg \min_a \ell(\hat{x}_{1:H}^{\text{mean}}(a), g), \quad a_{\text{sto}}^{\star} = \arg \min_a \mathbb{E}_m[\ell(x_{1:H}^{(m)}(a), g)].$$

The deterministic selector optimizes one mean trajectory; the distribution-aware selector minimizes expected failure, and a CVaR variant minimizes the tail. Common random numbers across candidates remove sampling noise from the comparison between actions.

2.7 Evaluation protocol: replaying the simulator to obtain ground truth

Scoring a stochastic world model against single realized trajectories conflates a wrong distribution with an unlucky draw. From any state we instead take an *exact MuJoCo fork*, resample the hidden physical mode m , and roll N independent realizations, producing samples from the true conditional $p(x_{t+H} \mid x_t, a_{t:t+H})$. Against this ground truth we report the energy score, a strictly proper scoring rule,

$$\text{ES}(F, y) = \mathbb{E}_{X \sim F} \|X - y\| - \frac{1}{2} \mathbb{E}_{X, X' \sim F} \|X - X'\|,$$

and its skill relative to deterministic LeWM, $\text{skill} = 1 - \text{ES}(\text{model})/\text{ES}(\text{det})$. Calibration is reported as covariance relative Frobenius error, quantile ECE, and central-interval coverage. A *sampling-attribution gate* additionally scores every model against its own predictive mean, so a gain cannot be a disguised bias correction. All intervals are paired scene-cluster bootstraps; splits are immutable and episode-disjoint; datasets, checkpoints and statistics artifacts carry SHA-256 provenance. Physical structure in the latents is measured with frozen ridge probes fit on train episodes only.

Table 1: Summary of measured effects. Each row compares two models, or two ways of planning with one model, on the same data. Prediction is reported as improvement in energy score (higher is better); control as percentage points of tasks solved in simulation. Intervals are computed by resampling whole scenes, so scenes that happen to be easy or hard cannot inflate the result. Read the baselines carefully: the two prediction rows and the step-by-step control row compare against deterministic LeWM, while the commitment row compares two ways of planning with *our own* model and the last row compares two idealized planners. In short: predictions improve reliably; the only direct success-rate comparison against LeWM is the step-by-step row, and its interval includes zero, so we claim no success-rate gain over LeWM.

Level	Comparison	Effect	95% CI	Assessment
Prediction	residual model vs deterministic LeWM, 10 steps ahead, 3 runs	+34.8%	[+32.0, +37.3]	improvement
Prediction	residual model vs its own average prediction	+20.2%	[+19.2, +21.3]	improvement is in the range, not the average
Control	planning over the range of outcomes vs over the average, commitment task	+3.81 pp	[+1.76, +5.86]	improvement
Control	residual model vs deterministic, ordinary step-by-step control	+2 pp	[−3, +7]	no effect (interval includes zero)
Ceiling	the same two planners given <i>perfect</i> physics, one shove per scene	+22.0 pp	[+14.0, +30.0]	headroom

3 Results

Every number below compares our model against the unmodified deterministic LeWM on the same data, so differences cannot come from a different evaluation set. Table 1 collects the measured effects with their uncertainty; Table 2 gives the data, training and evaluation settings they rest on. We report where the method works, where it does not, and how far the best result still sits below what perfect knowledge of the physics would allow.

Reading the numbers. Four quantities recur.

- **Energy score** measures how well a *set* of predicted futures matches what really happened: lower is better, and it can only be made small by predicting the right spread as well as the right average.
- **Skill** restates the same thing as a percentage improvement over the deterministic model, so 0% means no better and +30% means the error shrank by roughly a third.
- **Coverage** asks whether a stated 90% range actually contains the truth 90% of the time; a value below 0.90 means the model is overconfident.
- **Percentage points (pp)** are used for control: a move from 52% to 56% of tasks solved is +4 pp, not +8%.

3.1 Distributional prediction accuracy

Where the improvement comes from It matters whether a gain comes from the sophisticated part of a method or from something trivial, so Table 3 builds the model up one step at a time on a

Table 2: Experimental setup. Everything the numbers below depend on. Table 3 comes from a single training run and so carries no intervals; the multi-step FetchPush evaluation uses three runs, and all control results report intervals.

Stage	Configuration
Prediction data	OGBench cube quadruple-stack, 1,000 episodes / 383,766 transitions, eight hidden physical settings (object mass, surface friction, gripper-pad friction) that stay fixed within an episode; the train/validation/test split is fixed in advance at 8/1/1 and never shares an episode between parts
Frozen nominal	Unmodified LeWM retrained on this task, 30 epochs, batch size 64; held-out prediction error 0.00227
Residual heads	10 epochs, batch size 64; all heads share the same normalization statistics; the flow is integrated in 8 steps for evaluation and 4 during planning
One-step evaluation	2,048 held-out predictions, 16 sampled futures per prediction, one training run
Fork evaluation	FetchPush, equal numbers of slippery and sticky surfaces, 10 steps ahead, 3 training runs
Deployable state variant	Operates on the robot’s ordinary 28-number observation; it never reads friction labels or hidden simulator state
Commitment evaluation	1,024 previously unseen scenes, each run on both surfaces (2,048 episodes), reported across 8 shards
Planner	Plans by simulating $N=8$ possible futures per candidate action, reusing the same random draws across candidates so they are compared fairly; scores either the average outcome or the worst tail. The reference study in Table 5 uses 100 unseen scenes and no learned model

four-cube stacking task: no uncertainty at all, then noise of the right size but with nothing learned, then noise whose size depends on the situation, then the flow, then the flow with memory. The base model underneath was retrained on this task first and predicts it well (prediction error 0.00227, $25\times$ lower than the released checkpoint), which matters because it means what is left over for our model to explain is genuine randomness rather than the base model’s own mistakes. This is a single training run, so we read the *ordering* of the steps from it, not the exact sizes.

The ordering is informative, and two parts of it count against our own method. First, over a single step the flow is not better than the far simpler situation-dependent Gaussian noise, which achieves the lower energy score (9.55 against 10.04). What the flow does better is capture how the different parts of the state go wrong *together* rather than one at a time — the "errors in step" column, where lower is better (0.774 against 0.874). That kind of structure matters over longer rollouts, which is the test in the next paragraph. Second, all four models state ranges that are slightly too narrow: a nominal 90% range contains the truth about 81% of the time, short of the 0.87–0.93 band we committed to in advance. They are mildly overconfident.

Multi-step evaluation (FetchPush, three seeds) The harder and more realistic test asks the model to predict ten steps ahead, on a task where the table is equally often slippery or sticky. Here the residual model improves the energy score by +34.8% over deterministic LeWM, and by +20.2% over its *own* average prediction (Table 1). The second comparison is the important one: a model can

Table 3: Where the one-step improvement comes from. Four-cube stacking task, frozen base model, one training run, 2,048 held-out predictions — single-run numbers, so no intervals are given. Energy score (lower is better) is in the units of the model’s own state space, so only the comparisons between rows are meaningful; skill restates it as an improvement over row 0, whose score of 14.1 follows from that definition. The last column compares each model against its *own* average prediction: a positive value means the model gains from describing a range of outcomes, not from shifting its average.

Model	ES ↓	Skill ↑	Errors in step ↓	90% cov.	vs own average ↑
0 Deterministic LeWM	14.10	0	—	—	—
1 Fixed-size noise (nothing learned)	10.84	+23.1%	0.906	0.809	0.255
2 Situation-dependent Gaussian noise	9.55	+32.3%	0.874	0.793	0.258
3 Residual flow, no memory	10.04	+28.8%	0.774	0.814	0.258
4 Residual flow with memory	9.83	+30.3%	0.769	0.811	0.259

look better simply by shifting its average, and this one does not — the improvement comes from representing the range of possible outcomes. In absolute terms at one seed, the deterministic energy score is 2.676, against 1.825 for situation-dependent Gaussian noise, 1.706 for the residual flow and 1.703 for the flow with memory; the flow’s improvement over deterministic is 0.971 ([0.851, 1.098]). Adding memory changes nothing here (difference 0.0023, interval $[-0.045, +0.047]$), which makes sense: these predictions start before the robot touches the block, so no evidence about the surface has arrived yet.

Second environment: PushT On a short one-epoch training run in a second, 2D task, the flow again captures which errors move in step better than the Gaussian noise model (0.476 against 0.840) and states more honest ranges (0.863 against 0.817 coverage, target 0.90), while being fractionally worse on a finer calibration measure (0.0263 against 0.0241). Because the underlying LeWM is frozen throughout, none of this can degrade the original model’s own predictions.

3.2 Transfer of predictive gains to closed-loop control

Standard control: no measurable effect In ordinary control, the robot re-plans after every step. Our model predicts better than the deterministic one in this setting too (energy score 0.335 against 0.353, a +5.2% improvement, and +11.0% against its own average, both intervals clear of zero), yet the fraction of tasks it completes does not change: 20% against 18%, with an interval of $[-3, +7]$ pp that includes zero. A second controller working in the latent space gave the same non-result. The reason is that constant re-planning already absorbs surprises, and for most of the run a slippery table and a sticky table call for the same next action — so knowing the range of outcomes buys nothing.

Why committing changes the picture The benefit appears when the robot must decide *before* it can find out what the surface is like. Consider a single push: the arm commits one shove and then lets go, and the attempt counts as a success if the block passes within a tolerance τ of the goal. A slippery table makes the block overshoot; a sticky one makes it stop short. Averaging those two outcomes produces a phantom trajectory that lands neatly on the goal even though neither real outcome does —

Table 4: Commitment task, simulator success. 1,024 previously unseen scenes, each run under both a slippery and a sticky surface (2,048 episodes). The model never sees the friction, at training or at test time. The baseline is *our own model* collapsed to a single average prediction — not the original LeWM. Both rows therefore use the same trained model and differ only in how the planner reasons, which is what isolates the benefit of reasoning over a range. The lower two rows are reference points measured on the same scenes, not competing methods.

Selector	Overall	Low friction	High friction
Planning on our model’s average prediction	52.34%	80.57%	24.12%
Planning on the range of outcomes	56.15%	85.74%	26.56%
Effect (95% CI)	+3.81 pp		
[+1.76, +5.86]	+5.17 pp	+2.44 pp	
<i>Reference bounds on the same contexts</i>			
Same model trained with friction labels	+6.45 pp		
[+4.59, +8.30]	—	—	
Planner given the true outcome distribution	+17.09 pp	—	—

so a planner that trusts the average picks a shove that fails both ways. A planner that reasons over the range of outcomes rejects that shove and looks for one that works for both surfaces.

3.3 Committing under unknown physics, and how much is left on the table

Table 4 gives the result on that push task. The two rows being compared use exactly the same trained model and the same 1,024 starting scenes; the only difference is whether the planner picks its shove from the model’s average prediction or from the full range of predicted outcomes. The average-prediction row is not the original LeWM: it is our model with its uncertainty switched off, which is the stricter comparison, since any difference cannot be credited to the extra training. Reasoning over the range solves +3.81 pp more scenes, an improvement that holds in every one of the eight data shards, and it helps on both surfaces rather than trading one against the other. The model is never told what the friction is, at training time or at test time.

Two further rows put that gain in context. If the model is *given* friction labels while training, the gain roughly doubles to +6.45 pp — so removing the labels costs about 40% of the benefit, not all of it. And a planner allowed to consult the simulator’s true outcome distribution gains +17.09 pp, which is what a perfect version of our model would achieve.

How much is there to win in principle? To answer that without any learning at all, we replaced the model with the simulator itself. For each scene we try a dense library of candidate shoves, run each one under both frictions, and then compare two idealized planners that both have perfect physics and differ only in how they reason. The first picks the shove that looks best *on average* across the two surfaces; the second picks the shove least likely to fail on *either*. Table 5 scores them two ways: per episode, where each surface is judged separately, and per scene, the honest test for a commitment — one shove must work whichever surface nature turns out to have supplied.

Table 5: How much the task rewards this reasoning, using the simulator in place of a model. 100 previously unseen scenes. Neither row is a learned model: both planners query the simulator for the true outcome under each surface, and they differ only in whether they judge a shove by its average outcome or by its worst case. The table therefore bounds what this style of reasoning is worth in the task, independently of how well any model predicts. Friction lists the two possible surfaces as multipliers of the nominal value, and τ is how close to the goal counts as success. *Episode* scores each surface separately; *context* requires one committed shove to work on both, which is the situation the robot is actually in. At $0.25 \times / 2.5 \times$ and $\tau = 5$ cm a shove that is safe on both surfaces exists in 99% of scenes, and the range-aware planner finds it in 99 of 100 while the average-based planner succeeds in 77. A wider sweep showed this is a sweet spot: with more extreme frictions the two landing points are too far apart for any single shove to satisfy both, and with milder frictions the task becomes easy for either planner.

Friction	τ	Metric	Average-based oracle	Range-aware oracle	Gain (95% CI)
$0.25 \times / 2.5 \times$	5 cm	episode	88.5%	99.5%	+11.0 [+7.0, +15.0]
$0.25 \times / 2.5 \times$	5 cm	context	77.0%	99.0%	+22.0 [+14.0, +30.0]
$0.25 \times / 2.5 \times$	3 cm	context	69.0%	94.0%	+25.0 [+17.0, +34.0]
$0.2 \times / 3.0 \times$	5 cm	context	33.0%	58.0%	+25.0 [+17.0, +34.0]

The two idealized planners are separated by +22 pp: with perfect physics, thinking about the range of outcomes solves 99 of 100 scenes where thinking about the average solves 77. Our learned model captures roughly +3.8 pp of a comparable gap (Tables 4 and 5). The remaining problem is therefore a modelling one — the task clearly rewards this kind of reasoning, and our kernel is not yet good enough to collect most of the reward.

3.4 Ablations and probing analysis

What the memory actually learns We designed the recurrent memory hoping it would work out which surface the robot is on and sharpen its predictions accordingly. Table 7 shows that it does something else. Feeding it real observed transitions does help — its predictions beat those of a memory that has been wiped, and the benefit survives out to 200 updates even though training only ever showed it two. But when we train a small classifier to read the surface type out of the memory’s internal state, it never does better than always guessing the most common surface, at any depth, for either base model, and for both linear and neural classifiers. So the memory is adapting to recent context, not identifying the hidden surface, and we describe it that way rather than as the filter we set out to build. The information is available and the representation is what fails: the friction is 99.6% recoverable from how far the block moved, yet after five observed steps the model narrows its predicted range by less than 0.05%.

3.5 Limitations and scope of the claims

- **A harder sliding task defeats the method.** In FetchSlide the block is struck once and slides freely to a stop, so the prediction must run 50 steps rather than 10. The task would reward our approach even more strongly than pushing — with perfect physics, average-based reasoning solves none of 64 scenes and range-aware reasoning solves 47.7% — but the learned prediction

Table 6: Ablations. Each row changes one thing and holds everything else fixed — same frozen base model, same normalization — so the difference can be attributed to that one factor. The base-noise row uses a separate set of scenes whose baseline success is 62%, so it is reported as a paired difference rather than an absolute rate.

Ablation	Result	Reading
Released base model vs retrained on this task	Leftover systematic error drops 1.11 $\rightarrow$ 0.05; typical size of the remaining randomness 0.494 $\rightarrow$ 0.028	Retraining the base model removes its systematic error, so what remains is genuine randomness
Right amount of noise, nothing learned	+23.1% of the roughly 30% total improvement	Over one step, getting the size of the uncertainty right matters more than its shape
Flow vs situation-dependent Gaussian, one step	Energy score 10.04 vs 9.55; errors in step 0.774 vs 0.874	The flow wins on which errors move in step, not on single-step accuracy
With vs without memory (Fetch-Push)	Difference 0.0023, interval $[-0.045, +0.047]$ — includes zero	Memory adds nothing before the robot has touched the object
Memory fed 1 $\rightarrow$ 200 real transitions	Improvement over a wiped memory falls from +0.24 to +0.08 but stays above zero throughout; the memory’s internal values remain in a healthy range	Stays useful far beyond the two updates seen in training; no special long-sequence training needed
Outcome fixed per episode vs re-randomized each step	+3.81 pp, against no effect when re-randomized (62.21% vs 62.45%, interval $[-1.71, +1.22]$)	An outcome must stay consistent for a whole episode; step-by-step randomness is not enough
Without vs with friction labels in training	+3.81 pp without labels vs +6.45 pp with them	Roughly 60% of the benefit survives when friction labels are removed

breaks down over that horizon (energy score 1.127 against 0.309 for the plain base model), and control succeeds in 0% of attempts against 3.91%. The unmodified LeWM degrades over the same horizon too (energy score rising from 0.856 at one step to 13.832 at fifty), so the failure is inherited from predicting far ahead, not caused by adding the residual.

- **The model does not learn from what it observes.** As reported above, watching real transitions does not make its predictions more confident, even though the information needed is plainly present in the data.
- **The control result depends on the task, and is measured against our own model.** The +3.81 pp gain compares reasoning over a range against reasoning over the average *within* our model; we have not yet run the commitment task with deterministic LeWM as the planner, so we cannot state a success-rate improvement over LeWM. The one place we do compare the two directly — ordinary step-by-step control — shows no effect. The learned model also stays far short of what perfect physics allows, and its stated ranges are slightly too narrow.

Table 7: What the memory helps with, and what it fails to learn. Top row: how much better the model predicts when its memory has seen real transitions than when the memory is wiped (all intervals above zero, so the benefit is real). Bottom row: how accurately the hidden surface type can be read out of that same memory. Always guessing the most common surface scores 0.18, so the memory carries no usable information about which surface it is on. The memory itself stays numerically healthy at all depths, so this is not a case of the recurrent state degrading.

Replayed updates	1	2	5	12	30	50	100	150	200
Memory-vs-reset skill $\uparrow$	+ .24	+ .27	+ .26	+ .24	+ .17	+ .09	+ .16	+ .05	+ .08
Mode-probe accuracy	.16	.22	.14	.12	.12	.12	.18	.06	.08

3.6 Reproducibility and provenance

Every scene used for testing was held back from training, and no episode ever appears in more than one of the training, tuning and testing sets. Intervals are obtained by resampling whole scenes rather than individual predictions, so a handful of unusually easy or hard scenes cannot make an effect look larger than it is. Which comparisons would count as a success was written down before the experiments were run, and each dataset, trained model and set of normalization statistics carries a fingerprint, so a reported number can be traced back to the exact files that produced it. We also record the action the robot was actually commanded to take at every step, so a planning result cannot be confused with an execution error. The idealized planners that consult the simulator are kept strictly apart from the models we claim are deployable: those read only the robot’s own observations. Code, configurations and result files are on the `latent-residual-flow` branch.

References

- [1] D. Ha and J. Schmidhuber, "Recurrent world models facilitate policy evolution," *Advances in Neural Information Processing Systems*, vol. 31, 2018.
- [2] D. Hafner et al., "Learning latent dynamics for planning from pixels," in *International Conference on Machine Learning*, 2019.
- [3] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi, "Dream to control: Learning behaviors by latent imagination," in *International Conference on Learning Representations*, 2020.
- [4] J. Schrittwieser et al., "Mastering Atari, Go, chess and shogi by planning with a learned model," *Nature*, 2020.
- [5] G. Zhou, H. Pan, Y. LeCun, and L. Pinto, "DINO-WM: World models on pre-trained visual features enable zero-shot planning," arXiv:2411.04983, 2024.
- [6] Y. LeCun, "A path towards autonomous machine intelligence," OpenReview, 2022.
- [7] M. Assran et al., "Self-supervised learning from images with a joint embedding predictive architecture," in *CVPR*, 2023, pp. 15619–15629.
- [8] A. Bardes et al., "Revisiting feature prediction for learning visual representations from video," 2024.

- [9] M. Assran et al., "V-JEPA 2: Self-supervised video models enable understanding, prediction and planning," arXiv:2506.09985, 2025.
- [10] L. Maes, Q. Le Lidec, D. Scieur, Y. LeCun, and R. Balestrieri, "LeWorldModel: Stable end-to-end joint-embedding predictive architecture from pixels," arXiv:2603.19312, 2026.
- [11] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," in *International Conference on Learning Representations*, 2023.
- [12] X. Liu, C. Gong, and Q. Liu, "Flow straight and fast: Learning to generate and transfer data with rectified flow," in *International Conference on Learning Representations*, 2023.
- [13] A. Tong et al., "Improving and generalizing flow-based generative models with minibatch optimal transport," *Transactions on Machine Learning Research*, 2024.
- [14] M. S. Albergo, N. M. Boffi, and E. Vanden-Eijnden, "Stochastic interpolants: A unifying framework for flows and diffusions," *Journal of Machine Learning Research*, 2025.
- [15] C. Chi et al., "Diffusion policy: Visuomotor policy learning via action diffusion," *International Journal of Robotics Research*, 2024.
- [16] K. Black et al., " $\pi 0$: A vision-language-action flow model for general robot control," arXiv:2410.24164, 2024.
- [17] M. P. Deisenroth and C. E. Rasmussen, "PILCO: A model-based and data-efficient approach to policy search," in *International Conference on Machine Learning*, 2011.
- [18] A. Nagabandi, K. Konoglie, S. Levine, and V. Kumar, "Deep dynamics models for learning dexterous manipulation," in *Conference on Robot Learning*, 2020.
- [19] S. Greydanus, M. Dzamba, and J. Yosinski, "Hamiltonian neural networks," in *Advances in Neural Information Processing Systems*, 2019.
- [20] M. Lutter, C. Ritter, and J. Peters, "Deep Lagrangian networks: Using physics as model prior for deep learning," in *International Conference on Learning Representations*, 2019.
- [21] J. Lanier et al., "Adapting world models with latent-state dynamics residuals," arXiv:2504.02252, 2025.
- [22] Y. Jiang, L. Zhao, M. Chen, and D. Balkcom, "FlowMo-WM: A world model with object momentum and hidden ambient drift," arXiv:2606.13817, 2026.
- [23] X. Zhang, Z. Xu, Y. Tao, Y. Wang, Y. She, and A. Boularias, "Learning visual feature-based world models via residual latent action," arXiv:2605.07079, 2026.